

Diffusion Models for Urban Traffic Time Series: A Unified Framework for Probabilistic Forecasting and Pattern-Conditioned Imputation

Xiyang Liu

March 14, 2026

Abstract

Urban traffic management relies on accurate modeling of spatiotemporal sensor data, yet two fundamental challenges persist: probabilistic forecasting under uncertainty, and reconstruction of missing values caused by sensor failures. While these tasks are typically addressed with separate architectures, we propose a unified framework based on denoising diffusion probabilistic models that can handle both problems through conditional generation. We adapt CSDI — a diffusion model for time series imputation — with graph convolutional networks to capture spatial dependencies among traffic sensors, and compare it against TimeGrad, an autoregressive diffusion model for forecasting. Experiments on the METR-LA traffic dataset reveal a striking contrast: CSDI with graph convolutions achieves state-of-the-art imputation performance, reducing MAE by 35% over traditional methods (1.98 mph vs. 3.06 mph), with an ablation study confirming that spatial modeling is critical (removing graph convolutions more than doubles error). However, for forecasting, both diffusion models underperform a simple persistence baseline, suggesting that short-term traffic predictability is dominated by recent observations and that these architectures may require task-specific modifications. Our results demonstrate that diffusion models offer a powerful unified approach for traffic imputation while highlighting the need for improved forecasting designs and more comprehensive baselines in future work.

1 Introduction

Rapid urbanization has placed unprecedented strain on transportation infrastructure in many major cities across the globe. Intelligent traffic management thus becomes a critical challenge that produces many benefits if tackled properly. Urban traffic planning is data-driven, and traffic sensor networks — large arrays of traffic sensors deployed across highway systems — provide tremendous insights into how congestion patterns vary temporally and spatially [1]. These rich data streams capture complex, nonlinear interactions across space and time, offering the potential to model, predict, and ultimately mitigate urban congestion. However, two persistent and coupled challenges are present: inherent uncertainty in forecasting future traffic states, and frequent corruption of observed data due to sensor malfunction, communication loss, or environmental interference [1].

To tackle these two challenges, many existing time-series and deep-learning approaches treat them as separate problems and create different models with distinct architectures to handle them. For forecasting, models rely on autoregressive designs that propagate information causally forward in time. Popular empirical methods use discriminative models, including graph-based spatio-temporal neural networks such as STGCN (Spatio-Temporal Graph Convolutional Networks) and T-GCN (Temporal Graph Convolutional Network) that model dependencies across sensors and time and

make predictions [2, 3]. For imputation, methods such as Multivariate Imputation by Chained Equations (MICE) are the most prominent, which make use of regression models to fill in missing values one at a time [4]. At present, most work in this field treats imputation and forecasting as two separate stages performed sequentially. Predictive models with this architecture overlook a fundamental property of traffic patterns: both tasks are ultimately learning a coherent distribution over traffic dynamics at different times and locations. In principle, a model that can capture this underlying distribution is adaptable to both forecast and imputation tasks.

A compelling framework that unifies both problems is denoising diffusion probabilistic models (DDPMs), which can learn the underlying spatio-temporal distribution governing traffic flows in the sensor array data stream. By adding noise to original data and learning to invert the added noise, DDPMs can in principle approximate any given distribution to within arbitrary error [5]. Under this framework, forecasting is equivalent to conditional generation of a future sequence of samples given a known history sequence. Imputation can be casted as conditional inpainting with observed data fixed and missing data denoised. Furthermore, DDPM offers uncertainty-aware prediction and imputation: multiple plausible reconstructions can be sampled from the traffic load distribution, which permits uncertainty quantification.

DDPMs have been proven powerful in time series analysis. Lin et al. provided a comprehensive survey of diffusion methods applied to various forecasting tasks including conditional generation [6]. Conditional Score-based Diffusion models for Imputation (CSDI) proposed by Tashiro et al. is proven successful at time series imputation tasks [7]. However, few studies target DDPM-based analysis specifically in traffic time series modeling, which this work aims to achieve.

The goal of this project is to demonstrate how diffusion models can address two structurally distinct problems in urban traffic analysis probabilistic multi-step forecasting, which requires conditioning on past observations and generating a distribution for future states, and pattern-conditioned imputation, which requires acausal, constraint-guided reconstruction of missing components. The project’s scope is limited to exploring how DDPM performs on METR-LA — a traffic data analysis dataset containing the traffic network information in Los Angeles city from March to Jun 2012 [8]. Depending on project timeline, there may be extensions, as discussed in the next section.

2 Methodology

This section describes the two diffusion-based models used for traffic imputation and forecasting: CSDI and TimeGrad. We first review denoising diffusion probabilistic models (Section 2.1), then detail the adaptation of CSDI for imputation (Section 2.2) and TimeGrad for forecasting (Section 2.3). Finally, we explain how graph convolutions are integrated to capture spatial dependencies (Section 2.4).

2.1 Denoising Diffusion Probabilistic Models

Denoising diffusion probabilistic models (DDPMs) [9] are latent variable models that learn to reverse a gradual noising process. Given a data distribution $q(\mathbf{x}^{(0)})$, the forward process is a fixed Markov chain that adds Gaussian noise over K steps:

$$q(\mathbf{x}^{(k)}|\mathbf{x}^{(k-1)}) = \mathcal{N}(\mathbf{x}^{(k)}; \sqrt{1 - \beta_k} \mathbf{x}^{(k-1)}, \beta_k \mathbf{I}), \quad (1)$$

where β_k is a variance schedule. We can sample $\mathbf{x}^{(k)}$ directly from $\mathbf{x}^{(0)}$:

$$\mathbf{x}^{(k)} = \sqrt{\bar{\alpha}_k} \mathbf{x}^{(0)} + \sqrt{1 - \bar{\alpha}_k} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (2)$$

with $\alpha_k = 1 - \beta_k$ and $\bar{\alpha}_k = \prod_{s=1}^k \alpha_s$.

The reverse process is learned: a model $\epsilon_\theta(\mathbf{x}^{(k)}, k)$ predicts the noise ϵ that was added. Training minimizes the simple mean-squared error:

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}^{(0)}, k, \epsilon} \left[\|\epsilon - \epsilon_\theta(\mathbf{x}^{(k)}, k)\|^2 \right]. \quad (3)$$

After training, sampling starts from pure noise $\mathbf{x}^{(K)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and iteratively applies the reverse transition for $k = K, \dots, 1$:

$$\mathbf{x}^{(k-1)} = \frac{1}{\sqrt{\alpha_k}} \left(\mathbf{x}^{(k)} - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k}} \epsilon_\theta(\mathbf{x}^{(k)}, k) \right) + \sigma_k \mathbf{z}, \quad (4)$$

where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is optional stochastic noise (omitted for deterministic sampling).

2.2 CSDI for Imputation

CSDI extends DDPMs to handle missing values [7]. We adapt it to multivariate spatio-temporal data by replacing its Transformer backbone with a graph-enhanced U-Net that processes time and sensors jointly. CSDI explicitly models missing data via masking and conditioning, and its non-autoregressive nature allows parallel processing of all time steps. The U-Net architecture with graph convolutions enables it to capture both temporal and spatial patterns. Furthermore, CSDI improves 40–65% over existing probabilistic imputation methods with healthcare and environmental data on many performance metrics [7].

2.2.1 Model Architecture

Our CSDI model is built upon a spatio-temporal U-Net that operates on tensors of shape (B, T, N, F) , where B is batch size, T time steps, N sensors, and F features (here $F = 1$ for speed and the only feature is traffic speed). The U-Net consists of:

- An **input projection** that maps the F channels to an initial hidden dimension H_0 using a 3×3 convolution over time and sensors.
- **Encoder blocks**: each applies temporal downsampling (stride-2 convolution along time) followed by a graph convolution (Section 2.4) and stores a skip connection.
- A **bottleneck** with an additional graph convolution.
- **Decoder blocks**: each upsamples temporally, concatenates the corresponding skip connection, applies a graph convolution, and passes to the next level.
- An **output projection** that maps the final hidden representation back to F channels.

The U-Net accepts three input channels concatenated along the feature dimension: the noisy target, the conditioning represented as observed values with zeros for missing, and a mask indicating observed (1) vs missing (0). Thus $F_{\text{in}} = 3$.

2.2.2 Training

For imputation, we randomly mask a fraction p of the values in each training window. The mask $\mathbf{m} \in \{0, 1\}^{T \times N \times 1}$ is 1 for observed positions and 0 for missing. The conditioning $\mathbf{c}$ is the original data with missing positions set to zero. The target $\mathbf{x}^{(0)}$ is the original window.

During training, we sample a diffusion step $k \sim \mathcal{U}\{1, \dots, K\}$ and add noise to the target:

$$\mathbf{x}^{(k)} = \sqrt{\bar{\alpha}_k} \mathbf{x}^{(0)} + \sqrt{1 - \bar{\alpha}_k} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (5)$$

The model predicts the noise $\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_\theta(\mathbf{x}^{(k)}, k, \mathbf{c}, \mathbf{m})$. Because observed positions are already known, we compute the loss only on missing positions:

$$\mathcal{L}_{\text{imp}} = \mathbb{E} \left[\left\| (\mathbf{1} - \mathbf{m}) \odot (\boldsymbol{\epsilon} - \hat{\boldsymbol{\epsilon}}) \right\|^2 \right] / \mathbb{E}[\|\mathbf{1} - \mathbf{m}\|_1]. \quad (6)$$

This ensures the model focuses on learning to impute the missing entries.

2.2.3 Sampling

At test time, we start from pure noise $\mathbf{x}^{(K)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and iteratively apply the reverse process while clamping observed values to their known quantities. For each reverse step $k = K, \dots, 1$:

1. Predict noise using the model: $\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_\theta(\mathbf{x}^{(k)}, k, \mathbf{c}, \mathbf{m})$.
2. Compute the previous sample $\mathbf{x}^{(k-1)}$ via the reverse transition using deterministic DDIM [10] for speed.
3. Re-apply the observed values: $\mathbf{x}^{(k-1)} = \mathbf{x}^{(k-1)} \odot (\mathbf{1} - \mathbf{m}) + \mathbf{c} \odot \mathbf{m}$.

After K steps, $\mathbf{x}^{(0)}$ contains the imputed window.

2.2.4 CSDI for Forecasting

Although the primary design of CSDI targets imputation, it can be adapted for forecasting by treating the entire future window as missing. The input to the model is the concatenation of:

- **Noisy future:** the target future window with added diffusion noise,
- **Conditioning:** a representation of the observed past (e.g., the last observed value repeated across the future steps),
- **Mask:** a tensor of zeros for all future positions, indicating that they are entirely missing.

The model is then trained to denoise the future window conditioned on the past. During sampling, the reverse process starts from pure noise and iteratively reconstructs the future while the past conditioning remains fixed. This approach effectively turns the imputation model into a forecasting one, though it lacks the autoregressive nature of dedicated forecasting models like TimeGrad.

In our experiments, we used this adaptation as a baseline to compare against the autoregressive TimeGrad. The architecture and hyperparameters are identical to those described for imputation (Section 2.2), with the only difference being the mask and conditioning strategy.

2.3 TimeGrad for Forecasting

TimeGrad [11] is an autoregressive diffusion model for multivariate time series forecasting. It combines a recurrent neural network (RNN) that encodes the past with a diffusion decoder that generates one future step at a time. The RNN provides a dynamic context for each future step, which is more flexible than conditioning on a fixed window.

2.3.1 Model Architecture

Our TimeGrad implementation consists of:

- An **LSTM encoder** that processes the observed past window $\mathbf{p}_{1:\tau}$. Here, denote N the number of sensors, $\tau = 12$ the window length, and just one feature. The encoder is applied after flattening the sensor dimension into the batch dimension to share weights across sensors: input shape $(B \cdot N, \tau, 1)$. It produces a final hidden state $\mathbf{h}_\tau \in \mathbb{R}^{B \cdot N \times H}$.
- A **diffusion decoder** – a small MLP – that takes as input:
 - the noisy value at the current future step $\mathbf{x}_t$ (flattened to $(B \cdot N, 1)$),
 - the current hidden state $\mathbf{h}_{t-1}$ (flattened),
 - an embedding of the diffusion timestep k .

The decoder outputs the predicted noise $\hat{\epsilon}_t$ of the same shape. The decoder is shared across all steps and all sensors.

2.3.2 Training with Teacher Forcing

Given a batch of past windows $\mathbf{p}_{1:\tau}$ and corresponding ground-truth future windows $\mathbf{f}_{1:T}$ where $T = 12$, we:

1. Encode the past with the LSTM to obtain initial hidden states $\mathbf{h}_\tau$ and cell states $\mathbf{c}_\tau$.
2. For each future step $t = 1, \dots, T$:
 - Sample a diffusion step $k \sim \mathcal{U}\{1, \dots, K\}$ and add noise to the ground-truth value $\mathbf{f}_t$:

$$\mathbf{f}_t^{(k)} = \sqrt{\bar{\alpha}_k} \mathbf{f}_t + \sqrt{1 - \bar{\alpha}_k} \boldsymbol{\epsilon}_t, \quad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

- Flatten sensors: $\tilde{\mathbf{f}}_t^{(k)} \in \mathbb{R}^{B \cdot N \times 1}$.
- Predict noise with the decoder using the current hidden state $\mathbf{h}_{t-1}$:

$$\hat{\epsilon}_t = \text{Decoder}(\tilde{\mathbf{f}}_t^{(k)}, \mathbf{h}_{t-1}, k).$$

- Compute the MSE loss for this step: $\ell_t = \|\hat{\epsilon}_t - \epsilon_t\|^2$.
- **Teacher forcing**: update the RNN hidden state using the true $\mathbf{f}_t$ as input:

$$(\mathbf{h}_t, \mathbf{c}_t) = \text{LSTM}(\tilde{\mathbf{f}}_t, (\mathbf{h}_{t-1}, \mathbf{c}_{t-1})).$$

3. The total loss is the average over future steps: $\mathcal{L}_{\text{fore}} = \frac{1}{T} \sum_{t=1}^T \ell_t$.

Backpropagation is performed through both the diffusion decoder and the RNN.

2.3.3 Sampling

We generate multiple probabilistic trajectories by repeating the following procedure:

1. Encode the past to obtain $\mathbf{h}_\tau, \mathbf{c}_\tau$.

2. For each future step $t = 1, \dots, T$:
 - Start from pure noise $\mathbf{x}_t^{(K)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
 - Iteratively denoise for $k = K, \dots, 1$ using a DDIM scheduler with 50 inference steps. At each sub-step, the decoder predicts noise using the current hidden state $\mathbf{h}_{t-1}$.
 - After denoising, obtain $\mathbf{x}_t^{(0)}$, the forecast for step t .
 - Update hidden state with the generated value: feed $\mathbf{x}_t^{(0)}$ into the LSTM to obtain $\mathbf{h}_t, \mathbf{c}_t$.
3. Collect the generated sequence $\{\mathbf{x}_t^{(0)}\}_{t=1}^T$.

By running this process multiple times with different noise initializations, we obtain an ensemble of forecasts, from which we can compute mean predictions and uncertainty intervals.

2.4 Graph Convolution

Traffic data is inherently graph-structured. Spatial dependencies among sensors are captured via graph convolutions to improve predictions. We use the road network adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ provided with the METR-LA dataset, where entries are normalized and thresholded.

2.4.1 Graph Convolution Layer

We employ a Graph Convolutional Network (GCN) layer [12]:

$$\mathbf{X}' = \sigma \left(\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{X} \mathbf{W} \right), \quad (7)$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, $\tilde{\mathbf{D}}$ is the degree matrix of $\tilde{\mathbf{A}}$, and $\mathbf{W}$ is a learnable weight matrix. We add the identity matrix to the road adjacency matrix to account for self-loops.

2.4.2 Batching for Multiple Graphs

In both CSDI and TimeGrad, at certain points we have a tensor of shape (B, T, N, H) that we want to process with a graph convolution. Since the adjacency is the same for every batch element and time step, we flatten the batch and time dimensions and construct a block-diagonal edge index with appropriate offsets. This allows the GCN to operate on a single large graph of $B \cdot T \cdot N$ nodes, where each sub-graph is a copy of the sensor graph.

3 Experimental Setup

This section details the dataset preparation, hyperparameter choices, baseline methods, and planned ablations used to evaluate the proposed models.

3.1 Dataset and Preprocessing

We evaluate on the METR-LA traffic dataset [8], which contains 4 months of traffic speed readings (from March 1 to June 30, 2012) collected from 207 sensors on Los Angeles County highways. The data is aggregated at 5-minute intervals, resulting in a time series of length 34,272 per sensor.

Data Alignment. The dataset provides both the sensor graph (adjacency matrix) and the speed time series. We first align the sensors present in the graph with those in the data; all 207 sensors appear in both. The data is then reordered to match the graph ordering, ensuring consistent indexing for graph convolutions.

Normalization. Each sensor’s speed values are normalized to zero mean and unit variance using statistics computed on the training set. The per-sensor mean μ_n and standard deviation σ_n are saved to allow forecasts to be transformed back to original units (mph).

Temporal Split. The data is split chronologically: 70% for training, 10% for validation, and 20% for testing. This yields 23,990 training steps, 3,427 validation steps, and 6,855 test steps.

Window Generation. Sliding windows of length 24 are extracted with stride 1. For forecasting, each window provides a past sequence of 12 steps and a future target of 12 steps. For imputation, the full 24-step window is used as the target, and a random mask with corruption rate $p = 0.15$ is applied to create observed and missing positions.

3.2 Hyperparameters

Table 1 summarises the key hyperparameters used for training both models. Where a range is indicated, we performed a small grid search over the validation set to select the best value.

Parameter	CSDI	TimeGrad
Input / output length	12 / 12	12 / 12
Hidden dimensions	[64, 128, 256]	64 (RNN)
Number of diffusion steps (DDPM)	1000	1000
Number of sampling steps (DDIM)	50	50
Noise schedule	linear	linear
Learning rate	1×10^{-4}	1×10^{-4}
Batch size	16	16
Optimizer	AdamW	AdamW
Epochs	50	50
Graph convolution type	GCN	None (per-sensor MLP)
Corruption rate (imputation)	0.15	N/A

Table 1: Hyperparameter settings for CSDI and TimeGrad.

For CSDI, the U-Net architecture uses three down/up sampling levels. For TimeGrad, the RNN is a single-layer LSTM with hidden size 64; the diffusion decoder is a 3-layer MLP with 128 hidden units and SiLU activation. Time embeddings are injected via a small MLP.

3.3 Baselines

We compare our proposed methods against standard baselines for forecasting and imputation. All baselines are evaluated on the same test windows and using the same masking procedure as the proposed models, with metrics computed only on the artificially masked positions.

Forecasting Baselines

- **Persistence:** The forecast for all future steps is set to the last observed value of each sensor. This simple baseline often performs competitively for short-term traffic forecasting and serves as a sanity check.

Additional forecasting baselines, including historical average and linear regression, are planned for future work to provide a more comprehensive comparison.

Imputation Baselines

- **Forward fill:** Missing values are replaced with the last observed value in the same sensor if available; otherwise the next observed value is used.
- **Linear interpolation:** Missing segments are linearly interpolated between the nearest observed values in the same sensor.

4 Results

4.1 Forecast

Table 2 presents the forecasting results on the METR-LA test set. Surprisingly, the simple persistence baseline achieves the best performance across all metrics, with a normalized MAE of 0.2841 (5.45 mph) and normalized RMSE of 0.5327 (10.22 mph). Both deep learning methods perform substantially worse: TimeGrad yields a normalized MAE of 0.6733 (12.92 mph), while CSDI configured for forecasting produces the highest errors with a normalized MAE of 1.0738 (20.60 mph). The poor performance of the neural approaches relative to persistence suggests that either the models are not capturing the temporal dynamics effectively, or that the short-term predictability of traffic speeds is dominated by the most recent observation. However, these conclusions remain preliminary due to the absence of standard forecasting baselines such as historical average and linear regression, which are noted as future work.

Method	MAE (norm)	RMSE (norm)	MAE (mph)	RMSE (mph)
Persistence	0.2841	0.5327	5.45	10.22
TimeGrad	0.6733	1.0047	12.92	19.28
CSDI (forecast)	1.0738	1.3045	20.60	25.03

Table 2: Comparison of forecasting methods on the METR-LA test set. The (norm) columns denote errors computed on normalized data, while the (mph) columns report errors in the original speed units. Historical average and linear regression baselines are not yet implemented and will be included in future work.

4.2 Imputation

Figure 1 visualizes two example imputation windows from the test set, comparing the true values against the imputed means and 95% confidence intervals produced by CSDI. The model successfully reconstructs the missing segments with reasonable uncertainty estimates, capturing both the overall trend and local variations.

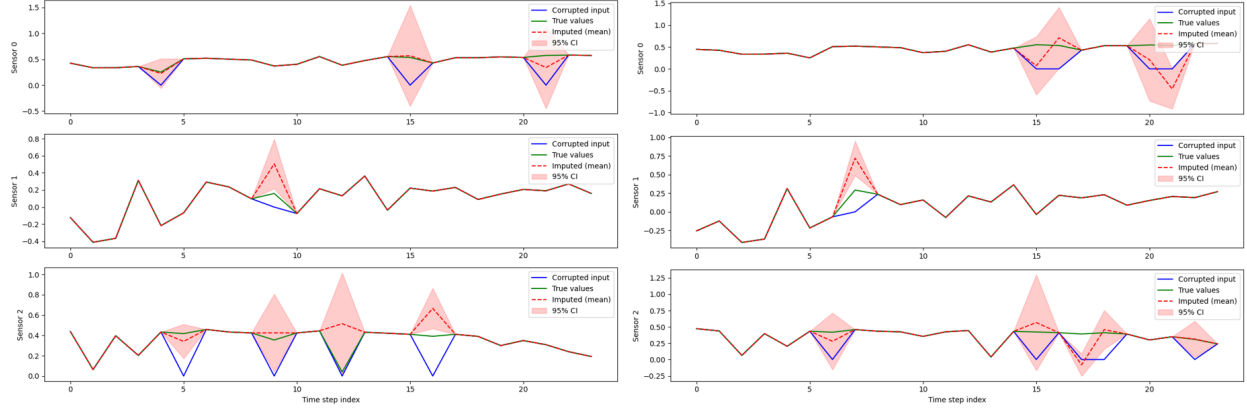


Figure 1: Two example imputation windows showing true values, imputed mean, and 95% confidence interval.

Table 3 provides a quantitative comparison of imputation methods under a 15% random corruption pattern. CSDI with graph convolutions achieves the lowest errors, with a normalized MAE of 0.1032 (1.98 mph) and normalized RMSE of 0.1709 (3.28 mph). This represents a substantial improvement over the traditional methods: forward fill achieves a normalized MAE of 0.1594 (3.06 mph), and linear interpolation achieves 0.1610 (3.09 mph). The relative improvement of CSDI over the best traditional baseline is approximately 35% in MAE and 47% in RMSE, demonstrating the value of learned generative models for traffic imputation.

Method	MAE (norm)	RMSE (norm)	MAE (mph)	RMSE (mph)
Forward fill	0.1594	0.3257	3.06	6.25
Linear interpolation	0.1610	0.3293	3.09	6.32
CSDI (imputation)	0.1032	0.1709	1.98	3.28

Table 3: Comparison of imputation methods on METR-LA test set (corruption rate 15%). Mean imputation is not yet implemented and will be added in future work.

4.3 Ablation Study: Effect of Graph Convolutions

To assess the contribution of spatial modeling, Table 4 compares CSDI with and without graph convolutional layers. This is implemented by setting the adjacency matrix to the identity matrix. Removing graph convolutions degrades performance dramatically: normalized MAE increases from 0.1032 to 0.2434 (from 1.98 mph to 4.67 mph), and normalized RMSE increases from 0.1709 to 0.4030 (from 3.28 mph to 7.73 mph). This more than twofold increase in error confirms that modeling spatial dependencies among sensors is critical for accurate imputation in traffic networks.

Configuration	MAE (norm)	RMSE (norm)	MAE (mph)	RMSE (mph)
CSDI with graph conv	0.1032	0.1709	1.98	3.28
CSDI without graph conv	0.2434	0.4030	4.67	7.73

Table 4: Ablation study: effect of removing graph convolutions from CSDI imputation with 15% corruption. Removing spatial information significantly degrades performance.

5 Discussion

The results reveal a striking contrast between forecasting and imputation. While CSDI with graph convolutions excels at imputation — outperforming traditional methods by a wide margin — both neural approaches fail to beat a simple persistence baseline in forecasting. This disparity can be attributed to the difference in available information: imputation benefits from bidirectional context (past and future observations) and strong spatial correlations, whereas forecasting must rely solely on past data. The ablation study confirms that spatial information is indeed the key driver of imputation performance, as removing graph convolutions leads to severe degradation.

The poor forecasting results also highlight potential mismatches between model design and task requirements. CSDI, originally developed for time series imputation, may not be directly applicable to forecasting without architectural modifications. TimeGrad, while designed for probabilistic forecasting, may require more extensive hyperparameter tuning or longer training periods to outperform simple baselines on this dataset. The high errors suggest that these models may be overparameterized for the relatively predictable nature of short-term traffic speeds, or that they fail to capture the temporal dependencies effectively.

Several limitations should be considered when interpreting these findings. First, the absence of statistical baselines (historical average, ARIMA, vector autoregression) makes it difficult to assess whether the poor forecasting performance is specific to these neural models or reflects the inherent predictability of the data. Second, the forecasting evaluation uses point metrics (MAE, RMSE) that may not fully capture the strengths of probabilistic models; proper scoring rules for distributions would provide a more complete assessment. Third, the imputation experiments use a fixed corruption pattern; performance may vary with different missing data mechanisms or rates.

Future work will address these limitations by incorporating additional baselines, conducting systematic hyperparameter optimization, and evaluating imputation under various missing data scenarios. For forecasting, we plan to explore architectures specifically designed for spatiotemporal prediction, such as graph neural networks with temporal convolutions or attention mechanisms. The strong imputation results also motivate extending CSDI to other domains with spatial structure, such as energy systems or environmental monitoring, where missing data is common and spatial dependencies are informative.

References

- [1] B. Medina-Salgado, E. Sánchez-DelaCruz, P. Pozos-Parra, and J. E. Sierra, “Urban traffic flow prediction techniques: A review,” *Sustainable Computing: Informatics and Systems*, vol. 35, p. 100739, 2022.
- [2] B. Yu, H. Yin, and Z. Zhu, “Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting,” *arXiv preprint arXiv:1709.04875*, 2017.
- [3] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, and H. Li, “T-gcn: A temporal graph convolutional network for traffic prediction,” *IEEE transactions on intelligent transportation systems*, vol. 21, no. 9, pp. 3848–3858, 2019.
- [4] S. Van Buuren and K. Groothuis-Oudshoorn, “mice: Multivariate imputation by chained equations in r,” *Journal of statistical software*, vol. 45, pp. 1–67, 2011.

- [5] G. Li, C. Cai, and Y. Wei, “Dimension-free convergence of diffusion models for approximate gaussian mixtures,” *arXiv preprint arXiv:2504.05300*, 2025.
- [6] L. Lin, Z. Li, R. Li, X. Li, and J. Gao, “Diffusion models for time-series applications: a survey,” *Frontiers of Information Technology & Electronic Engineering*, vol. 25, no. 1, pp. 19–41, 2024.
- [7] Y. Tashiro, J. Song, Y. Song, and S. Ermon, “Csdi: Conditional score-based diffusion models for probabilistic time series imputation,” *Advances in neural information processing systems*, vol. 34, pp. 24 804–24 816, 2021.
- [8] H. He, “California traffic network datasets: Metr-la, pems-bay, pems04 and pems08 for traffic speed and flow analysis,” 2025. [Online]. Available: <https://dx.doi.org/10.21227/j49q-ch56>
- [9] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
- [10] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” *arXiv preprint arXiv:2010.02502*, 2020.
- [11] K. Rasul, C. Seward, I. Schuster, and R. Vollgraf, “Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting,” in *International conference on machine learning*. PMLR, 2021, pp. 8857–8868.
- [12] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *arXiv preprint arXiv:1609.02907*, 2016.